在任何 control 传输里都省不掉（SETUP 之后无论有没有 Data，都以一个反方向零长 DATA1 收尾），而 **status 的方向永远与 Data 阶段相反**——Data 方向由 SETUP 的 D 位定，status 就取反方向；因此 `GET_DESCRIPTOR`（Control Read）的 status 是 OUT，`SET_CONFIGURATION`（No-data、D=0）的 status 是 IN。

对回本项目：`control()` 如何走这三相. `control()` 在 ch0 上按 SETUP→(Data)→STATUS 依次调用 `channel_xfer()`：SETUP 用 `PID_SETUP` 带 8 字节请求；数据阶段按方向以 `in=1/0` 反复搬运 mps 大小的块并翻转 PID；STATUS 用反方向、`len=0`、DATA1。挂在 Hub 后面的低速设备，这三相里的每一个事务又要各自拆成一对 SSPLIT/CSPLIT（下一小节）——所以“读 18 字节描述符”在总线上其实是 3 个 IN 事务、每个再走 split 两拍，这也解释了为什么必须逐包翻转、少取一包就会读到缓冲区里的残值。

表 7.5 是识别一个 Hub 后面的键盘所依次发出的 control 请求。

表 7.5: 从 Hub 端口到可用键盘的请求序列

对象	请求	bmRequestType	作用
Hub	`SET_FEATURE(PORT_POWER)`	`0x23`	给下游端口上电，之后最多等 500 ms 连接去抖
Hub	`GET_STATUS`	`0xa3`	端口状态：bit 0 连接、bit 1 使能、bit 8 供电、bit 9 低速、bit 10 高速
Hub	`SET_FEATURE(PORT_RESET)`	`0x23`	复位端口，设备进入默认状态、地址为 0
设备 0	`GET_DESCRIPTOR(Device, 8)`	`0x80`	只取 8 字节，得到端点 0 的最大包长
设备 0	`GET_DESCRIPTOR(Device, 18)`	`0x80`	完整设备描述符：class、VID、PID
设备 0	`SET_ADDRESS(n)`	`0x00`	分配唯一地址；把父 Hub、端口、速度记入 `usb_route[n]`
设备 *n*	`GET_DESCRIPTOR(Config, 9)`	`0x80`	得到 `wTotalLength`
设备 *n*	`GET_DESCRIPTOR(Config, total)`	`0x80`	全部接口和端点，挑出 Boot Keyboard
设备 *n*	`SET_CONFIGURATION`	`0x00`	激活配置，端点开始可用
接口	`SET_PROTOCOL(boot)`	`0x21`	让键盘用固定的 8 字节 Boot 报告格式
接口	`SET_IDLE(0)`	`0x21`	只在按键状态变化时才回报告

先取 8 字节再取 18 字节是 USB 枚举的惯例：在知道端点 0 的最大包长之前，只能假定它是最小的 8 字节，而设备描述符第 7 字节正好告诉我们真实值。端口状态的例子：真机日志里 Hub port 3 是 `0x301`（供电、低速、已连接），复位后变成 `0x303`（多了“使能”）；根 Hub 和端口上电之后并不是只读一次状态，而是在复位枚举前最多等 500 ms，让 `PORT_CONNECTION` 稳定下来，冷启动较慢的 Hub、无线键盘接收器和低速键盘都靠这段去抖时间。如果等满之后仍然是 `0x100`，含义是控制器只看到 `PORT_POWER`，线上并没有设备的电气连接——此时 HID 驱动和 ch6 都还没有参与，不该往协议层去找原因。根 Hub 和嵌套的 Hub 用同一套请求递归扫描，深度限制为两层，防止一个异常的拓扑让启动无限递归。扫描到的端点属性按 USB 地址记录在 `usb_ep0_mps[]` 里，Hub、低速键盘和鼠标不再共用一个全局的端点 0 包长。

把这一整套判断画成控制流就是图 7.8。它有五个阶段，值得注意的不是步骤本身，而是三处分支判据：class 9 决定要不要扫描下游端口，端口状态决定这个端口是否继续，class/subclass/protocol 决定这个接口是不是 Boot Keyboard。递归那一跳（子 Hub 回到端口扫描）和低速那一支（`LSPDDEV` 加 split 事务）是真机上最容易出错的两处——前者决定嵌套 Hub 能不能被发现，后者决定 EP0 的每个包要不要单独走一遍 SSPLIT/CSPLIT。

`SET_PROTOCOL(boot)` 很关键。HID 设备本来用“报告描述符”描述自己的报告格式，解析它需要一个不小的解释器；而 Boot 协议是 BIOS 时代为了不解析报告描述符而规定的固定格式，所有键盘都必须支持。用它，驱动只需理解图 7.9 这 8 个字节。

最后，`usb_device_register()` 把设备交给 USB 总线，总线按 class/subclass/protocol 匹配到 `usbhid-keyboard` 驱动，调用 `usbkbd_probe()`：发送上面两个 HID 请求，分配每设备的 `usbkbd_state` 和 URB，注册 `input_dev`，然后第一次提交 URB。

枚举与配置的软件函数调用链. 图 7.10回答的是“这些报文由哪一个 C 函数发起、设备何时进入通用设备模型，以及键盘的 interrupt-IN 请求最终在哪里启动”；它与 §7.4.9 热插拔一节的总线时序图（图 7.17）互为表里——那张图讲“USB 线上先后发送了什么”，这张图讲“这些报文由谁发起”。图中同一行的浅灰等宽字是这一步对应的串口日志，把函数调用与真机实际输出逐行对齐。启动代码先注册 USB 总线与类驱动，随后注册 DWC2